UNIVERSITY OF CENTRAL PUNJAB

Operating System - Lab

GRADED LAB – 02

Name		Roll No.	
Section	E4	Date	11/02/2026

Time allowed: 40 minutes Total marks: 10

Plagiarism Policy

- This is an individual assessment. All code must be written by you during the lab session.
- Copying code from another student, sharing your code, or allowing others to copy it is plagiarism and is treated as cheating by both parties.
- Using AI tools, online solutions, websites, messaging apps, or unauthorized notes to generate or obtain answers is not allowed.
- Code that is identical or suspiciously similar (including identical variable names, comments or mistakes) will be treated as plagiarism.
- You may be asked to explain your code. If you cannot explain it, you will receive zero marks for that question.
- Any student found guilty will receive zero marks in this graded lab, and the case will be reported according to university rules.

Task: Race Reporter (10 Marks)

Write a C program race.c that starts two programs at the same time, lets them race, and then reports which one finished first and how each one ended.

The program takes two integers from the command line:

```
./race <t1> <t2>
```

Requirements

1. **Input check.** The program needs exactly two arguments, and both must be integers from 1 to 9. If not, print a usage or error message and exit with status 1. No child may be created in this case.
2. **Two children, running together.** The parent creates two children, Runner A and Runner B. Both must be created before the parent waits for either of them, so that they run at the same time. If fork() fails, print an error using perror() and exit.
3. **What each child does.** Each child first prints one line with its name, PID and PPID. Then it turns itself into the sleep program:
    - Runner A runs sleep t1 using **execlp()** (list form).
    - Runner B runs sleep t2 using **execvp()** (vector form).
4. **When exec fails.** If exec returns, the child must print an error using perror() and terminate with exit status **99**. It must never continue into the parent's code.
5. **The parent reports in finishing order.** The parent calls wait() exactly twice. After each call it prints which runner finished (1st or 2nd), that runner's PID, and how it ended:
    - normal exit with code 99: print "exec FAILED"
    - any other normal exit: print the exit code
    - killed by a signal: print "killed by a signal"

    The parent must work out which runner finished from the PID returned by wait(), not by assuming an order.
6. **Winner.** After both children are reaped, the parent prints "Winner: Runner A" or "Winner: Runner B", meaning the runner that was reaped first.
7. **No zombies, no orphans.** Both children must be reaped before the parent ends.

Sample runs

```
$ ./race 3 1
Runner A: PID=4101, PPID=4100
Runner B: PID=4102, PPID=4100
Finished 1st: Runner B (PID 4102), exit code 0
Finished 2nd: Runner A (PID 4101), exit code 0
Winner: Runner B

$ ./race 2 5
Runner A: PID=4110, PPID=4109
Runner B: PID=4111, PPID=4109
Finished 1st: Runner A (PID 4110), exit code 0
Finished 2nd: Runner B (PID 4111), exit code 0
Winner: Runner A
```

```
$ ./race 0 5
Error: both values must be between 1 and 9
```

The two lines starting with "Runner" may appear in either order, because the scheduler decides who runs first.

Test your program with ./race 3 1, ./race 2 5, ./race 4 and ./race 0 5. To test the exec failure, temporarily change "sleep" in one call to a name that does not exist and check that the parent reports "exec FAILED" for that runner. Change it back before you submit.

Marking Scheme

Criterion	Marks
Argument validation; exit(1) with message and no fork on bad input	1
Both children created before any wait(); fork() failure checked	1.5
Children never fall through into parent code (exec failure path ends with exit(99))	1
Correct exec calls: execlp() with list ending in NULL, execvp() with array ending in NULL, argv[0] set properly	2
exec failure reported with perror() and status 99	1
Two wait() calls; runner identified from the returned PID; printed in finishing order; winner correct	2
Status handled correctly with WIFEXITED, WEXITSTATUS, WIFSIGNALED (including the 99 case)	1
Comment: with t1 equal to t2 the winner cannot be predicted, since scheduling order is not fixed	0.5
Total	**10**